

TP12bis — Pipeline complet de bout en bout

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Ce TP remerge la première partie de la formation (Git, Python, Ansible, YANG, Jinja2, Vault, ZDD, tests, diagnostic) avant de passer aux modules Cisco. Depuis le TP2, `.github/workflows/netops-pipeline.yml` tourne avec des jobs `validate/test/deploy` en placeholder, pendant que chaque brique réelle a été construite séparément. Ce TP leur donne du vrai contenu — et remplace au passage le validateur YANG écrit à la main (TP6) par `yangson`, une vraie bibliothèque, mise à l'épreuve sur **quatre scénarios réels**, testés directement sur le pipeline GitHub Actions (commit → push → observation dans l'onglet Actions), pas en local :

1. **Ouverture d'une nouvelle agence** avec une faute de frappe — bloquée à l'étape `VALIDATE`
2. **Modification d'un site existant** avec une valeur invalide — bloquée à l'étape `VALIDATE`
3. **Un bug dans le template** de configuration — détecté par le déploiement ZDD lui-même (rollback du canary)
4. **Un VLAN dupliqué entre deux interfaces actives** — le seul type d'erreur que le job `post-check` séparé peut détecter, invisible à tous les étages précédents

Important : ce TP réutilise autant que possible ce qui existe déjà (`tests/pre_checks/`, `tests/post_checks/`, `templates/interface_config.j2`, `ansible.cfg`, `yang/nexacorp-interfaces.yang`). Ne recréez pas un fichier si l'instruction ne le demande pas explicitement.

Prérequis

- TP1 à TP12 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour3-tp12bis
```

Étape 2 — Installer yangson

```
pip install yangson pyyaml
```

Étape 3 — Créer le descripteur “yang-library”

Le modèle YANG du TP6 (yang/nexacorp-interfaces.yang) reste **strictement inchangé**. yangson a simplement besoin d’un fichier séparé qui déclare quel module charger (norme RFC 7895) :

```
cat > yang/yang-library.json << 'EOF'
{
  "ietf-yang-library:modules-state": {
    "module-set-id": "1",
    "module": [
      {
        "name": "nexacorp-interfaces",
        "revision": "",
        "namespace": "urn:nexacorp:interfaces",
        "conformance-type": "implement"
      }
    ]
  }
}
EOF
```

Étape 4 — Créer le script VALIDATE (remplace scripts/yang_to_config.py du TP6 dans le pipeline)

network_model.yaml (TP6) reste lui aussi **inchangé** (format dict, sites: siege: interfaces: [...]). Le script valide chaque site **séparément** contre le modèle plat — un appel yangson par site, ce qui évite tout besoin de restructurer le modèle ou la donnée (voir “Comprendre le TP” pour la raison technique) :

```
cat > scripts/yang_validate.py << 'EOF'
"""Etape VALIDATE du pipeline : valide chaque site de network_model.yaml
contre le modele YANG (yangson), un site a la fois, puis écrit un
host_vars/<site>.yaml par site valide."""
import os
import sys

import yaml
from yangson import DataModel
from yangson.exceptions import YangsonException

HERE = os.path.dirname(os.path.abspath(__file__))
MODEL_FILE = os.path.join(HERE, "..", "inventory", "network_model.yaml")
LIBRARY_FILE = os.path.join(HERE, "..", "yang", "yang-library.json")
MODULE_DIR = [os.path.join(HERE, "..", "yang")]
HOST_VARS_DIR = os.path.join(HERE, "..", "inventory", "host_vars")

def main():
    dm = DataModel.from_file(LIBRARY_FILE, MODULE_DIR)

    with open(MODEL_FILE) as f:
        data = yaml.safe_load(f)

    os.makedirs(HOST_VARS_DIR, exist_ok=True)
    for site_name, site_data in data["sites"].items():
        raw = {"nexacorp-interfaces:interface": site_data["interfaces"]}
        instance = dm.from_raw(raw)
        try:
```

```

        instance.validate()
    except YangsonException as e:
        print(f"VALIDATION ECHOUÉE [{site_name}] : {e}")
        sys.exit(1)

    host_vars_path = os.path.join(HOST_VARS_DIR, f"{site_name}.yaml")
    with open(host_vars_path, "w") as f:
        yaml.safe_dump({"interfaces": site_data["interfaces"]}, f, allow_unicode=True)
    print(f"OK [{site_name}] : {len(site_data['interfaces'])} interface(s) validees")

print(f"\nVALIDATION OK : {len(data['sites'])} site(s) conformes au modele YANG.")

if __name__ == "__main__":
    main()
EOF

python3 scripts/yang_validate.py

```

Résultat attendu : VALIDATION OK : 3 site(s) conformes au modele YANG. et inventory/host_vars/*.yaml régénérés.

Étape 5 — Généraliser deploy_zdd.yaml

Le deploy_zdd.yaml du TP10 cible hosts: agence1,siege,agence2 — une liste **codée en dur**. On généralise la phase “rolling” pour qu’elle suive le groupe d’inventaire réel, agence1 restant le canary explicite :

```

cat > playbooks/deploy_zdd.yaml << 'EOF'
---
- name: "Canary : deploiement pilote sur agence1"
  hosts: agence1
  gather_facts: no
  any_errors_fatal: true
  tasks:
    - name: "Sauvegarder et deployer la configuration ({ inventory_hostname })"
      ansible.builtin.template:
        src: "{{ playbook_dir }}/../templates/interface_config.j2"
        dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
        backup: yes
        register: render_result

    - name: "Post-check et rollback automatique en cas d'echec ({ inventory_hostname })"
      block:
        - name: "Verifier que la configuration generee respecte les invariants attendus"
          ansible.builtin.assert:
            that:
              - "'ntp server' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
              - "'banner motd' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
              - "'hostname' + inventory_hostname in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
            fail_msg: "Post-check echoue sur {{ inventory_hostname }} : configuration invalide"
            success_msg: "Post-check reussi sur {{ inventory_hostname }}"

      rescue:
        - name: "ROLLBACK : restaurer la configuration precedente sur {{ inventory_hostname }}"
          ansible.builtin.copy:
            src: "{{ render_result.backup_file }}"
            dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
            remote_src: yes
            when: render_result.backup_file is defined
        - name: "Arreter tout le deploiement suite a l'echec sur {{ inventory_hostname }}"
          ansible.builtin.fail:
            msg: "Deploiement ZDD arrete : {{ inventory_hostname }} a echoue les post-checks. Rollback effectue. Les sites suivants ne seront PAS modifies."

- name: "Rolling : reste des sites de l'inventaire, un par un"
  hosts: "all:!agence1"
  gather_facts: no
  serial: 1
  any_errors_fatal: true
  tasks:
    - name: "Sauvegarder et deployer la configuration ({ inventory_hostname })"
      ansible.builtin.template:
        src: "{{ playbook_dir }}/../templates/interface_config.j2"
        dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
        backup: yes
        register: render_result

    - name: "Post-check et rollback automatique en cas d'echec ({ inventory_hostname })"
      block:
        - name: "Verifier que la configuration generee respecte les invariants attendus"
          ansible.builtin.assert:
            that:
              - "'ntp server' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
              - "'banner motd' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
              - "'hostname' + inventory_hostname in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
            fail_msg: "Post-check echoue sur {{ inventory_hostname }} : configuration invalide"
            success_msg: "Post-check reussi sur {{ inventory_hostname }}"

      rescue:
        - name: "ROLLBACK : restaurer la configuration precedente sur {{ inventory_hostname }}"
          ansible.builtin.copy:
            src: "{{ render_result.backup_file }}"
            dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
            remote_src: yes
            when: render_result.backup_file is defined
        - name: "Arreter tout le deploiement suite a l'echec sur {{ inventory_hostname }}"
          ansible.builtin.fail:
            msg: "Deploiement ZDD arrete : {{ inventory_hostname }} a echoue les post-checks. Rollback effectue. Les sites suivants ne seront PAS modifies."
EOF

```

hosts: "all:!agence1" désigne “tous les hôtes de l’inventaire, sauf agence1” — sans ce changement, une nouvelle agence ajoutée à l’inventaire serait silencieusement ignorée par le déploiement.

Étape 6 — Créer les points d’entrée pre-checks et post-checks

Au TP11, run_tests.yml exécutait pre-checks et post-checks l’un après l’autre dans le même playbook. Le pipeline a besoin de les déclencher à deux moments séparés — deux playbooks fins, qui reproduisent le même calcul de bilan et le même fail final que run_tests.yml (sans quoi un rescue interne masquerait un vrai échec) :

```
cat > playbooks/run_pre_checks.yml << 'EOF'
---
- name: Pre-checks avant déploiement
  hosts: all
  gather_facts: no
  tasks:
    - name: Exécuter les pre-checks
      ansible.builtin.include_tasks: "{{ playbook_dir }}/../tests/pre_checks/pre_check_site.yml"

    - name: Afficher le rapport de pre-checks pour ce site
      ansible.builtin.debug:
        msg: "{{ item.phase | upper }} [{{ 'OK' if item.passed else 'ECHEC' }}] {{ item.name }}"
      loop: "{{ test_results }}"
      loop_control:
        label: "{{ item.name }}"

    - name: Calculer le bilan des pre-checks pour ce site
      ansible.builtin.set_fact:
        nb_tests: "{{ test_results | length }}"
        nb_echecs: "{{ test_results | selectattr('passed', 'equalto', false) | list | length }}"

    - name: Afficher le bilan des pre-checks
      ansible.builtin.debug:
        msg: "PRE-CHECKS {{ inventory_hostname }} : {{ nb_tests | int - nb_echecs | int }}/{{ nb_tests }} réussis"

    - name: Faire échouer le playbook si au moins un pre-check a échoué
      ansible.builtin.fail:
        msg: "{{ nb_echecs }}" pre-check(s) en échec sur {{ inventory_hostname }} : déploiement annulé"
      when: nb_echecs | int > 0
EOF

cat > playbooks/run_post_checks.yml << 'EOF'
---
- name: Post-checks après déploiement
  hosts: all
  gather_facts: no
  tasks:
    - name: Exécuter les post-checks
      ansible.builtin.include_tasks: "{{ playbook_dir }}/../tests/post_checks/post_check_site.yml"

    - name: Afficher le rapport de post-checks pour ce site
      ansible.builtin.debug:
        msg: "{{ item.phase | upper }} [{{ 'OK' if item.passed else 'ECHEC' }}] {{ item.name }}"
      loop: "{{ test_results }}"
      loop_control:
        label: "{{ item.name }}"

    - name: Calculer le bilan des post-checks pour ce site
      ansible.builtin.set_fact:
        nb_tests: "{{ test_results | length }}"
        nb_echecs: "{{ test_results | selectattr('passed', 'equalto', false) | list | length }}"

    - name: Afficher le bilan des post-checks
      ansible.builtin.debug:
        msg: "POST-CHECKS {{ inventory_hostname }} : {{ nb_tests | int - nb_echecs | int }}/{{ nb_tests }} réussis"

    - name: Faire échouer le playbook si au moins un post-check a échoué
      ansible.builtin.fail:
        msg: "{{ nb_echecs }}" post-check(s) en échec sur {{ inventory_hostname }} : déploiement a ne pas considerer comme valide"
      when: nb_echecs | int > 0
EOF
```

hosts: all (pas nexacorp, qui n’existe pas dans l’inventaire réel — les groupes sont hub/spokes, tous deux enfants du groupe all).

Étape 7 — Corriger .yamllint

```
cat > .yamllint << 'EOF'
extends: relaxed
rules:
  line-length:
    max: 200
  truthy:
    allowed-values: ['true', 'false', 'yes', 'no']
EOF
```

extends: default (le réglage du TP2) est un jeu de règles très strict, jamais vraiment respecté par les fichiers écrits depuis. relaxed abandonne l'exigence de --- en tête de fichier et l'indentation au caractère près ; line-length: 200 laisse la place aux conditions Ansible longues ; truthy autorise yes/no, courants en Ansible (backup: yes, gather_facts: no).

Si l'erreur wrong new line character apparaît sur .yamllint lui-même (fins de ligne Windows) : `sed -i 's/\r$//' .yamllint`

Étape 8 — Ignorer generated_configs/

```
git rm -r --cached generated_configs/ 2>/dev/null
echo "generated_configs/" >> .gitignore
```

Ce dossier était censé être gitignored depuis le TP7 mais ne l'était jamais réellement. Les fichiers de sauvegarde (backup: yes) contiennent un horodatage avec des : — invalides dans un nom de fichier NTFS — que Windows/WSL remplace silencieusement par un caractère Unicode de substitution, ce qui casse actions/upload-artifact en CI si ces fichiers sont commités.

Étape 9 — Rendre l'inventaire dynamique exécutable

```
chmod +x inventory/dynamic_inventory.py
git update-index --chmod=+x inventory/dynamic_inventory.py
```

Sous Windows (Git Bash/WSL), core.fileMode est souvent à false — Git ignore alors silencieusement le `chmod +x` local au moment du commit, et le fichier reste non-exécutable dans le dépôt malgré tout, ce qui fait qu'Ansible tente de le parser comme un inventaire statique au lieu de l'exécuter (Error parsing host definition '""': No closing quotation). `git update-index --chmod=+x` force le bit exécutable **dans l'index Git lui-même**, indépendamment de ce réglage. Vérifiez avec `git ls-files -s inventory/dynamic_inventory.py` : le mode doit afficher 100755, pas 100644.

Étape 10 — Configurer les secrets GitHub

lab/.env et .vault_pass (TP8) sont tous les deux gitignored, donc absents sur un runner CI. Sur GitHub → Settings → Secrets and variables → Actions → New repository secret :

- API_TOKEN = la même valeur que celle chiffrée dans vault.yml (TP8)
- VAULT_PASSWORD = le contenu de votre .vault_pass local

Étape 11 — Écrire le pipeline GitHub Actions

```
cat > .github/workflows/netops-pipeline.yml << 'EOF'
name: NetOps Pipeline

"on":
  push:
    branches: [dev, staging, main, 'main-*', '*-jour*-tp*']
  pull_request:
    branches: [dev, staging, main, 'main-*']

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Installer yamllint
        run: pip install yamllint
      - name: Lint des fichiers YAML
        run: yamllint .

  validate:
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - name: Installer yangson
        run: pip install yangson pyyaml
      - name: Valider les donnees reseau contre le modele YANG
        run: python3 scripts/yang_validate.py
      - name: Publier les host_vars generes
        uses: actions/upload-artifact@v4
        with:
          name: host-vars
          path: inventory/host_vars/

  test:
    runs-on: ubuntu-latest
    needs: validate
    env:
      API_TOKEN: ${ secrets.API_TOKEN }
    steps:
      - uses: actions/checkout@v4
      - name: Recuperer les host_vars generes par le job validate
        uses: actions/download-artifact@v4
        with:
          name: host-vars
          path: inventory/host_vars/
      - name: Demarrer le lab
        run: docker compose -f lab/docker-compose.yml up -d --build
      - name: Attendre que les sites soient prêts
        run: sleep 3
      - name: Installer Ansible
        run: pip install ansible-core
      - name: Pre-checks reseau
        run: ansible-playbook --vault-password-file <(echo "${ secrets.VAULT_PASSWORD }") playbooks/run_pre_checks.yml

  deploy:
    runs-on: ubuntu-latest
    needs: test
    if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/heads/main-')
    environment: production
    env:
      API_TOKEN: ${ secrets.API_TOKEN }
    steps:
      - uses: actions/checkout@v4
      - name: Recuperer les host_vars generes par le job validate
        uses: actions/download-artifact@v4
        with:
          name: host-vars
          path: inventory/host_vars/
      - name: Demarrer le lab
        run: docker compose -f lab/docker-compose.yml up -d --build
      - name: Attendre que les sites soient prêts
        run: sleep 3
      - name: Installer Ansible
        run: pip install ansible-core
      - name: Preparer le dossier de sortie
        run: mkdir -p generated_configs
      - name: Deploiement ZDD (canary puis rolling)
        run: ansible-playbook --vault-password-file <(echo "${ secrets.VAULT_PASSWORD }") playbooks/deploy_zdd.yml
      - name: Publier les configurations generees
        uses: actions/upload-artifact@v4
        with:
          name: generated-configs
```

```

    path: generated_configs/
post-check:
  runs-on: ubuntu-latest
  needs: deploy
  if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/heads/main-')
  steps:
    - uses: actions/checkout@v4
    - name: Recuperer les host_vars generes par le job valide
      uses: actions/download-artifact@v4
      with:
        name: host-vars
        path: inventory/host_vars/
    - name: Recuperer les configurations generees par le job deploy
      uses: actions/download-artifact@v4
      with:
        name: generated-configs
        path: generated_configs/
    - name: Installer Ansible
      run: pip install ansible-core
    - name: Post-checks reseau
      run: ansible-playbook --vault-password-file <(echo "${ secrets.VAULT_PASSWORD }")> playbooks/run_post_checks.yml
EOF

```

Trois points à retenir sur ce workflow :

- La clé "on" : est **citée** — sinon YAML l'interprète comme le booléen true plutôt que comme le mot-clé GitHub Actions (le "Norway problem" du parsing YAML).
- deploy et post-check sont des jobs **séparés**, chacun sur une machine neuve — generated_configs/ et host_vars/ produits par un job ne sont visibles par le suivant qu'en les publiant explicitement (upload-artifact) et en les récupérant (download-artifact).
- La condition if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/heads/main-') permet à chaque étudiant de tester deploy/post-check sur sa propre branche main-<prenom>, sans toucher au vrai main partagé.

Étape 12 — Pousser l'état de référence (3 sites)

```

git add .
git commit -m "TP12bis : pipeline complet avec validation YANG réelle (yangson)"
git push -u origin <prenom>-<nom>-jour3-tp12bis

```

Ouvrez la pull request → dev, onglet **Actions**. **Résultat attendu** : lint/validate/test verts ; deploy/post-check "skipped" (pas sur main ni main-*).

Étape 13 — Scénario 1 : ouvrir une nouvelle agence

```

cat >> inventory/sites.yml << 'EOF'
- nom: agence3
  role: spoke
  api_port: 5004
EOF

python3 - << 'PYEOF'
content = open('inventory/network_model.yaml').read()
content += """  agence3:
    interfaces:
      - name: GigabitEthernet0/1
        description: "LAN utilisateurs agence3"
        enabled: up

```

```

        vlan-id: 4999
    - name: GigabitEthernet0/2
      description: "Reserve"
      enabled: down
      vlan-id: 20
"""
open('inventory/network_model.yaml', 'w').write(content)
PYEOF

```

Ajoutez le conteneur agence3-router à lab/docker-compose.yml (même patron que les 3 autres sites) :

```

cat > lab/docker-compose.yml << 'EOF'
services:
  siege-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: siege-router
    environment:
      - SITE_NAME=siege
      - API_TOKEN=${API_TOKEN}
    ports:
      - "5001:5000"
    networks:
      - nexacorp-lab

  agence1-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: agence1-router
    environment:
      - SITE_NAME=agence1
      - API_TOKEN=${API_TOKEN}
    ports:
      - "5002:5000"
    networks:
      - nexacorp-lab

  agence2-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: agence2-router
    environment:
      - SITE_NAME=agence2
      - API_TOKEN=${API_TOKEN}
    ports:

```

```

    - "5003:5000"
networks:
  - nexacorp-lab

agence3-router:
  build:
    context: ..
    dockerfile: lab/Dockerfile
  container_name: agence3-router
  environment:
    - SITE_NAME=agence3
    - API_TOKEN=${API_TOKEN}
  ports:
    - "5004:5000"
  networks:
    - nexacorp-lab

networks:
  nexacorp-lab:
    driver: bridge
EOF

```

Sans ce conteneur, test échouerait sur agence3 (connection refused) même une fois le VLAN corrigé — rien n'écouterait sur le port 5004.

```

git add inventory/ lab/docker-compose.yml
git commit -m "Ajout agence3 (avec une erreur volontaire de VLAN a corriger)"
git push

```

Résultat attendu dans Actions : validate échoue (vlan-id ... not in range), test/deploy/post-check ne tournent jamais.

Corrigez :

```

sed -i 's/vlan-id: 4999/vlan-id: 50/' inventory/network_model.yml
git add inventory/network_model.yml
git commit -m "Correction du VLAN agence3"
git push

```

Résultat attendu : tout repasse au vert, 4 sites.

Étape 14 — Scénario 2 : modifier un site déjà en service

```

sed -i '0,/enabled: up/{s/enabled: up/enabled: actif/}' inventory/network_model.yml
git add inventory/network_model.yml
git commit -m "Maintenance : désactivation temporaire (avec une erreur volontaire)"
git push

```

Résultat attendu : validate échoue (enabled ... expected admin-state(enumération)), le message référence directement le typedef admin-state du TP6.

Corrigez :

```
sed -i 's/enabled: actif/enabled: up/' inventory/network_model.yaml
git add inventory/network_model.yaml
git commit -m "Correction de la valeur enabled"
git push
```

Étape 15 — Pousser vers une branche main-<prenom> pour activer deploy/post-check

```
git checkout -b main-<prenom>
git push -u origin main-<prenom>
```

deploy et post-check ne s'exécutent que sur main ou main-* (voir Étape 11) — sans cette étape, ils resteront "skipped" indéfiniment.

Étape 16 — Scénario 3 : casser le template (test du rollback canary)

```
sed -i '/ntp server/d' templates/interface_config.j2
git add templates/interface_config.j2
git commit -m "Scenario 3 : bug volontaire dans le template"
git push
```

Résultat attendu : lint/validate/test verts (rien de faux dans la donnée elle-même), deploy échoue — le canary (agence1) déclenche son rollback interne (ntp server absent de la config générée), siege/agence2/agence3 ne sont **jamais** touchés (any_errors_fatal: true arrête tout après l'échec de la première play), et post-check ne tourne même pas (needs: deploy).

Corrigez :

```
sed -i '/banner motd/i ntp server {{ ntp_server }}' templates/interface_config.j2
git add templates/interface_config.j2
git commit -m "Correction du template"
git push
```

Étape 17 — Scénario 4 : tester le job post-check séparément

Le check "VLAN dupliqué entre interfaces actives" n'existe que dans tests/post_checks/post_checks (TP11) — ni YANG, ni le post-check interne de deploy_zdd.yml (qui ne vérifie que ntp server/banner motd/hostname) ne peuvent le détecter. On active la seconde interface d'agence2 avec le même VLAN que la première :

```
python3 - << 'PYEOF'
content = open('inventory/network_model.yaml').read()
content = content.replace(
    """  agence2:
    interfaces:
      - name: GigabitEthernet0/1
        description: "LAN utilisateurs agence2"
        enabled: up
        vlan-id: 40
```

```

        - name: GigabitEthernet0/2
          description: "Reserve"
          enabled: down
          vlan-id: 20""",
    """    agence2:
interfaces:
    - name: GigabitEthernet0/1
      description: "LAN utilisateurs agence2"
      enabled: up
      vlan-id: 40
    - name: GigabitEthernet0/2
      description: "Reserve"
      enabled: up
      vlan-id: 40""")
open('inventory/network_model.yaml', 'w').write(content)
PYEOF

```

```

git add inventory/network_model.yaml
git commit -m "Scenario 4 : VLAN duplique entre interfaces actives"
git push

```

Résultat attendu : lint/validate/test/deploy **tous verts** (chaque valeur est individuellement correcte — VLAN dans la plage, état valide) — seul post-check échoue, sur le test pas_de_vlan_duplique. C'est le seul scénario des quatre où ce job, et lui seul, détecte le problème.

Corrigez :

```

python3 - << 'PYEOF'
content = open('inventory/network_model.yaml').read()
content = content.replace(
    """    agence2:
interfaces:
    - name: GigabitEthernet0/1
      description: "LAN utilisateurs agence2"
      enabled: up
      vlan-id: 40
    - name: GigabitEthernet0/2
      description: "Reserve"
      enabled: up
      vlan-id: 40""",
    """    agence2:
interfaces:
    - name: GigabitEthernet0/1
      description: "LAN utilisateurs agence2"
      enabled: up
      vlan-id: 40
    - name: GigabitEthernet0/2

```

```

        description: "Reserve"
        enabled: down
        vlan-id: 20"""
    )
    open('inventory/network_model.yaml', 'w').write(content)
PYEOF

git add inventory/network_model.yaml
git commit -m "Correction : agence2 restauree (plus de VLAN duplique)"
git push

```

Étape 18 — Ouvrir la pull request finale

Ouvrir une pull request <prenom>-<nom>-jour3-tp12bis → dev, avec les 5 jobs visibles et verts dans l'onglet **Actions**.

Comprendre le TP

Pourquoi ce TP ferme cette partie de la formation. Tout ce qui a été appris depuis le TP1 n'existait jusqu'ici que comme des briques séparées. Ce TP les assemble en une seule chaîne que n'importe quel changement futur doit traverser — et c'est en la faisant vraiment tourner, pas seulement en l'écrivant, que sont apparus la plupart des problèmes ci-dessous.

yangson plutôt que le validateur maison. yang_to_config.py (TP6) ne vérifiait que ce qu'on avait explicitement codé. yangson applique automatiquement toutes les contraintes du .yang — y compris le typedef admin-state, déjà défini au TP6 sans jamais avoir été vérifié "en vrai".

Les quatre étages ne vérifient pas la même chose, et c'est voulu. VALIDATE vérifie chaque site individuellement contre un contrat de structure ; le post-check interne au déploiement vérifie les invariants d'un site juste après l'avoir déployé ; le job post-check séparé est le seul capable de comparer plusieurs interfaces **entre elles** au sein d'un même site, une fois tout le déploiement terminé. Une erreur peut être individuellement invisible à trois étages et n'apparaître qu'au quatrième — le VLAN dupliqué du Scénario 4 en est la preuve concrète.

Les bugs réels trouvés en testant, pas en imaginant. Chaque étage a révélé au moins un problème qui n'existait dans aucune documentation avant qu'on ne fasse tourner le vrai pipeline : un .yamllint jamais respecté, une clé on: interprétée comme un booléen, un inventaire dynamique dépourvu de son bit exécutable à cause de core.fileMode sous Windows, des groupes d'inventaire mal nommés, un conteneur Docker manquant pour un nouveau site, des host_vars non transmis entre jobs, des fichiers de sauvegarde Windows invalides jamais gitignored. Aucun de ces problèmes n'aurait été visible en lisant le code — seule l'exécution réelle, avec de vrais commits et un vrai pipeline, les a fait apparaître.

Validation

- ☐ lint/validate/test passent sur la pull request avec l'état de référence (3 sites)
- ☐ L'ajout d'agence3 avec un VLAN invalide arrête le pipeline à validate ; une fois corrigé, les 4 sites passent sans modification supplémentaire des playbooks
- ☐ La modification d'un site existant avec une valeur enabled invalide est détectée et bloquée par validate
- ☐ Sur une branche main-<prenom> : un template cassé fait échouer deploy (rollback du canary agence1), sans jamais toucher aux autres sites, et sans jamais lancer post-check
- ☐ Un VLAN dupliqué entre deux interfaces actives d'un même site passe lint/validate/test/deploy, et n'est détecté que par post-check
- ☐ Une pull request <prenom>-<nom>-jour3-tp12bis → dev est ouverte, avec les 5 jobs verts